Automated Testing In Jaltantra

Submitted in partial fulfillment of the requirements

for the degree of

Master of Technology

by

Ankush Mitra

Roll No: 23M0755

Supervisor(s):

Dr. Supervisor One

Dr. Supervisor Two (if any)

Department of Computer Science Engineering

Indian Institute of Technology Bombay

2025

Declaration

I declare that this written submission represents my ideas in my own words...

Signature Date

Name Roll No

Abstract

Jaltantra is a web-based system designed to optimize water distribution networks in rural areas. These networks are essential to ensure that water reaches every household. However, the existing setup in Jaltantra lacks proper testing. Initially, testing is performed manually, making it time-consuming, non-reproducible, and error-prone. In this project, we aim to introduce automated testing to improve the reliability, scalability, and maintainability of the system.

We implement unit testing using JUnit and Mockito to validate individual components such as cost calculation and flow logic. Integration tests verify the interactions between controllers, services, and databases using Spring Boot testing support. We also include security testing to ensure endpoints behave correctly under Spring Security and to validate password encryption and login flows. Tools such as `ReflectionTestUtils` are used to access and test private methods and fields.

To evaluate performance, we employ the open-source tool k6. We design and run smoke tests, load tests, stress tests, spike tests, and soak tests. These tests help us analyze system behavior under different types of traffic and monitor resource usage such as CPU and memory. Real-time metrics from Prometheus and Grafana dashboards enable us to observe JVM health, request latency, and other system parameters during these tests.

For automated vulnerability detection, we use OWASP ZAP. This tool identifies issues such as SQL injection, CSRF attacks, missing Content Security Policy (CSP) headers, and session mismanagement. Based on the findings, we implement corresponding fixes to strengthen the system's security posture.

To validate real-world workflows, we add end-to-end (E2E) testing using Selenium. This simulates user interactions across the entire application stack. All tests are made reproducible and maintainable through proper scripting and configuration practices.

Finally, we integrate these test suites into a continuous integration (CI) pipeline using GitHub Actions. With this setup, every code commit is automatically tested, providing immediate feedback and preventing regressions.

In total, we write over 105 automated tests. We refactor key classes using design patterns such as Template Method and Extract Method. As a result, we reduce the size of `BranchOptimizer.java` from over 6000 to approximately 3500 lines. This project substantially enhances the testability, structure, and overall quality of Jaltantra, making it more robust and future-ready.

Contents

List of Figures

List of Tables

Abbreviations and Notation

CI	Continuous Integration
CD	Continuous Deployment
E2E	End-to-End
API	Application Programming Interface
VU	Virtual User
GC	Garbage Collection
JVM	Java Virtual Machine
UI	User Interface
SLA	Service Level Agreement
IDE	Integrated Development Environment
ZAP	Zed Attack Proxy
k6	A modern load testing tool written in Go
HTTP	HyperText Transfer Protocol
CSV	Comma Separated Values
DSL	Domain-Specific Language

Notation:

- t — time in seconds
- n — number of virtual users (load level)
- $p(x)$ — percentile function (e.g., $p(95)$ is the 95th percentile)
- R — number of requests per second
- L — response latency in milliseconds

Chapter 1

Introduction

1.1 Introduction to Jaltantra

Water distribution systems play a crucial role in ensuring that clean water reaches communities efficiently. These systems are made up of interconnected components such as pipes, nodes, pumps, and reservoirs that transport water from treatment plants to households and other endpoints. The central goal is to meet demand at every node while optimizing for cost and maintaining adequate pressure throughout the network.

We work with **Jaltantra v7**, a web-based platform designed to optimize such water distribution networks [1]. The platform allows engineers to input various data elements including node elevations, pipe characteristics, demand profiles, and pump configurations. Optimization algorithms are then applied to determine cost-effective pipe diameters and flow configurations that satisfy both pressure and demand constraints.

Jaltantra supports both linear programming techniques for branched networks and nonlinear solvers for looped networks. It uses advanced solvers such as **Knitro** and **Baron**, wrapped via Python interfaces and executed with parallel processing to scale for large network instances. By supporting diverse network topologies and flexible objectives, the platform empowers engineers to design efficient and reliable water infrastructures.

1.2 System Architecture of Jaltantra

Jaltantra follows a modular web-based architecture designed to optimize both branched and looped water distribution networks. Users interact with a web interface to input network details and initiate optimization. The backend processes these requests by converting the input data into appropriate internal representations, performing validation, and invoking the optimization logic. For branched networks, optimization is performed using linear programming techniques [1], while for looped networks, external solvers are invoked through scripting interfaces. Optimization results are processed and returned to the frontend for visualization and analysis, ensuring a smooth and efficient workflow from input to output.

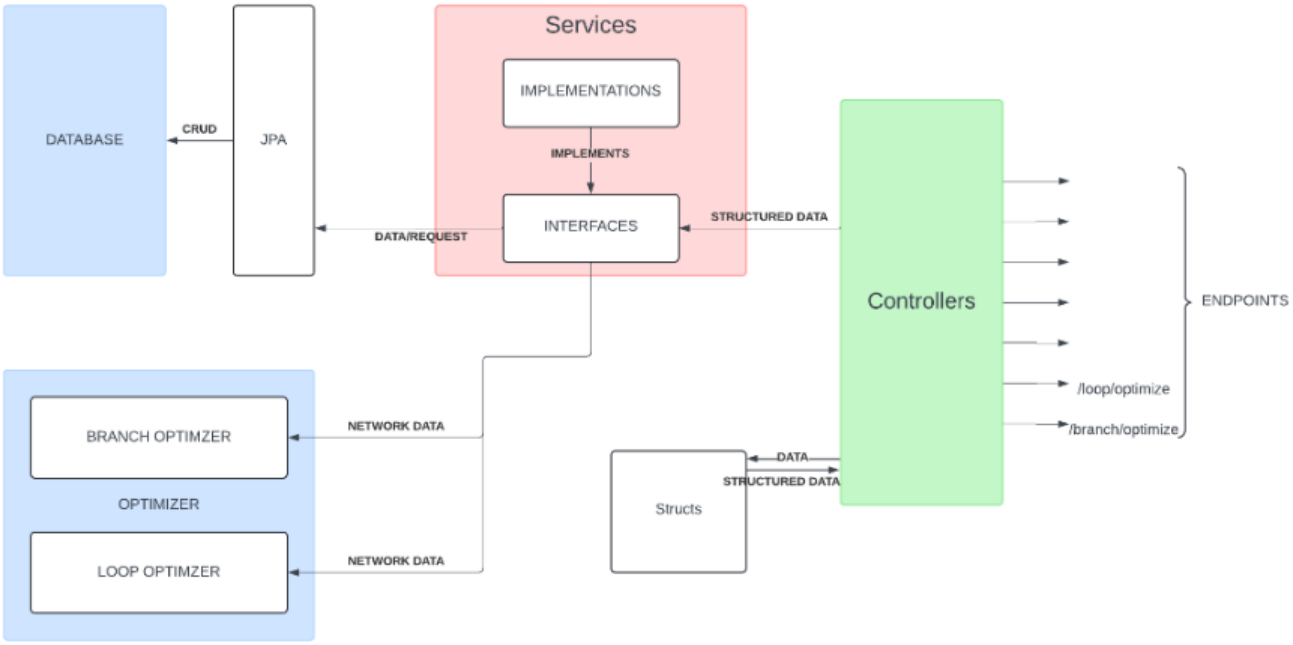

Figure 1.1: System Architecture of Previous Jaltantra Version (Jaltantra v7)

1.3 Motivation and Problem Statement

While Jaltantra offers strong optimization capabilities, its testing strategy has been largely manual. This approach introduces several limitations:

- Manual testing is time-consuming, error-prone, and inconsistent.
- There is no automated validation to catch regression issues after code changes.

- Security and performance aspects of the system are not systematically evaluated.
- New contributors face difficulty due to a lack of reproducible test scenarios.

To address these challenges, we aim to implement a comprehensive automated testing and quality assurance framework for Jaltantra.

1.4 Contributions

The following key contributions have been made to the Jaltantra platform:

- A complete and automated testing pipeline spanning unit, integration, and system-level tests has been introduced using `k6` [3].
- Over 100 test cases covering critical workflows and edge cases have been implemented using Selenium [6].
- Maintainability has been enhanced by systematically refactoring complex and redundant code.
- Platform reliability has been improved by integrating tests into the CI pipeline via GitHub Actions [5].
- Multiple security issues and performance inefficiencies have been identified and resolved through OWASP ZAP testing [4].

Chapter 2

Background & Literature Review

Before constructing an automated testing framework for Jaltantra, foundational software testing principles and the relevant tools were studied. This chapter outlines the motivations behind automated testing and code refactoring, followed by a summary of system-level evaluations and the technologies employed in Stage-II.

2.1 Stage-I Work

Software testing is recognized as a critical activity for verifying that software behaves correctly and satisfies user requirements. Historically, Jaltantra depended heavily on manual testing, which was time-consuming, difficult to reproduce, and prone to human error.

To overcome these drawbacks, automated testing was introduced. Such tests execute consistently on every code change, enabling early detection of regressions and reducing manual effort. Different testing strategies were adopted in this project:

- **Unit Testing:** Ensures the correctness of individual methods or components in isolation using JUnit [2].
- **Integration Testing:** Validates interactions among system layers such as controllers, services, and repositories using Spring Boot and MockMvc [2].
- **Mocking:** Dependencies like databases and external APIs were simulated using Mockito [2] to test modules independently.

The `BranchController` module manages essential operations such as optimization, file uploads, and user sessions. Manual testing of such features had previously led to inconsistencies and inefficiencies. With automation, consistent validation was achieved, testing duration was reduced, and rapid development feedback was ensured.

To increase code confidence and support test-driven development, unit and integration tests were implemented in Stage-I. These tests facilitated the verification of both functionality and structural coherence in the system's critical components.

Large files such as `BranchOptimizer.java`, which previously exceeded 6300 lines, posed challenges in terms of maintainability and testability. Refactoring was applied using standard design patterns like Template Method and Extract Method [1]. This restructuring resulted in modular code that supported focused and maintainable test cases.

2.2 Stage-II Work

The second phase of the project focused on enhancing system security, evaluating performance under stress conditions, and integrating these efforts into a Continuous Integration (CI) pipeline.

Security Testing

System authentication and access control mechanisms were assessed using Spring Security and MockMvc [2]. The following aspects were verified:

- Password encryption and secure storage.
- Restricted access to protected resources.
- Correct behavior of login and logout endpoints.

Furthermore, OWASP ZAP [4] was used to detect common security vulnerabilities such as SQL injection, CSRF attacks, and misconfigured security headers.

Performance Testing

To assess the system's scalability and responsiveness, the open-source performance testing tool k6 [3] was used. Several performance tests were executed:

- **Smoke Tests:** Confirmed that basic functionality remained intact.
- **Load Tests:** Simulated user traffic to measure response times.
- **Stress Tests:** Pushed the system beyond normal loads to observe failure points.
- **Spike and Soak Tests:** Measured system behavior during abrupt traffic increases and prolonged load durations.

Metrics such as CPU usage, memory consumption, and latency were collected using Prometheus [2] and visualized in Grafana dashboards [2].

End-to-End Testing

End-to-end user workflows were validated using Selenium [6]. These automated tests simulated user actions such as registration, login, file uploads, and optimization requests through the web interface, ensuring that subsystems functioned correctly when integrated.

CI Pipeline Integration

All test stages were integrated into a GitHub Actions workflow [5]. The CI pipeline automatically executed unit, integration, and end-to-end tests on each code commit or pull request. This automation improved development velocity and safeguarded the platform against regressions.

2.3 Tools and Technologies Used

The tools and frameworks listed below supported the implementation and validation processes:

- **Spring Boot:** Backend web framework for REST API development [2].
- **JUnit:** Framework for writing and executing unit tests in Java [2].
- **Mockito:** Used for mocking dependencies and isolating test logic [2].
- **MockMvc:** Facilitated API testing without launching a full server [2].

- **Selenium:** Supported browser automation for testing UI workflows [6].
- **k6:** Load testing tool used for scripting performance scenarios [3].
- **GitHub Actions:** Enabled CI workflows for automatic testing [5].
- **OWASP ZAP:** Detected web application vulnerabilities [4].
- **Prometheus:** Monitored runtime and JVM metrics [2].
- **Grafana:** Visualized monitoring metrics and system health .

Together, these tools enabled a secure, scalable, and maintainable testing pipeline for the Jaltantra system.

Chapter 3

Security Testing

Security is a critical aspect of any web-based system. A secure application ensures that users access only the data and functionality permitted to them. In Jaltantra, it is essential to test security-sensitive features such as login, access control, and password encryption to maintain system integrity.

Previously, the system lacked automated validation for the following:

- Public pages being accessible without authentication.
- Protected endpoints redirecting unauthenticated users.
- Proper loading of the login page.
- Secure hashing and verification of user passwords.

To address these issues, we introduce security-focused test cases using the Spring Security Test framework[2].

3.1 Spring Security Test Dependency

To enable security-related testing capabilities, we include the following dependency in the `pom.xml` file:

```
<!-- Spring Security Test -->
<dependency>
```

```
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <version>6.2.3</version>
    <scope>test</scope>
</dependency>
```

3.2 Types of Security Features Tested

We focus on verifying the following categories of security behavior:

- **Access Control:** Validate correct exposure and protection of endpoints.
- **Authentication:** Ensure that login forms and session creation behave correctly.
- **Authorization:** Confirm access to protected areas is granted only to authorized users.
- **Password Security:** Verify encryption and password matching mechanisms.

Below are Test Cases:

1. Test Public Endpoint Without Authentication

This test checks whether a public URL (such as the home page) remains accessible without login:

```
mockMvc.perform(MockMvcRequestBuilders.get("/"))
        .andExpect(status().isOk());
```

2. Test Private Endpoint Redirects to Login

We verify that unauthenticated users are redirected to the login page when accessing a protected route:

```
mockMvc.perform(MockMvcRequestBuilders.get("/dashboard"))
        .andExpect(status().is3xxRedirection())
        .andExpect(redirectedUrlPattern("**/login"));
```

3. Test Login Page Accessibility

This test confirms that the login page loads successfully and returns the correct view:

```
mockMvc.perform(MockMvcRequestBuilders.get("/login"))
        .andExpect(status().isOk())
        .andExpect(view().name("login"));
```

4. Test Login with Valid User

We simulate a successful login scenario using a mock user with valid credentials:

```
@WithMockUser(username = "user", roles = {"USER"})
mockMvc.perform(MockMvcRequestBuilders.post("/login")
         .param("username", "user")
         .param("password", "password"))
        .andExpect(status().isOk());
```

3.3 Password Encoding Test

It is essential to confirm that passwords are not stored in plain text. The following test checks encryption and matching using Spring's `PasswordEncoder`:

```
String rawPassword = "Password123";
String encodedPassword = passwordEncoder.encode(rawPassword);

assert passwordEncoder.matches(rawPassword, encodedPassword);
```

This confirms that:

- Raw passwords are securely encrypted.
- Encrypted passwords are matched correctly during login.

3.4 Conclusion

By introducing automated security tests with Spring Security Test, we ensure that:

- Access control across public and private routes behaves as expected.
- Login functionality operates reliably.
- Passwords are encrypted and verified in a secure manner.

These tests are integrated into the CI pipeline, ensuring continuous validation of the system's security posture on every code update. This significantly improves Jaltantra's reliability and resilience against common vulnerabilities.

Chapter 4

Automated Vulnerability Testing Using OWASP ZAP

4.1 What is Automated Vulnerability Testing?

Automated vulnerability testing is the process of detecting security weaknesses in a web application without manual intervention. Specialized tools simulate attacks that real-world attackers might attempt, such as SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

The key objectives of automated vulnerability testing are to:

- Identify common security flaws efficiently.
- Minimize manual testing overhead.
- Provide rapid feedback during development cycles.
- Enhance the application's overall security posture.

Rather than inspecting each input or HTTP request manually, automated scanners systematically explore the application and highlight risks based on known vulnerability signatures. These tools generally offer:

- **Passive Scanning:** Observing traffic without altering requests.
- **Active Scanning:** Actively attempting attacks by injecting payloads.

The tools produce structured reports containing vulnerability types, severity ratings (low/medium/high), and recommended remediations. Such reports enable developers to address security issues before deployment.

Table 4.1: Comparison of Automated Vulnerability Scanning Tools

Tool	**Advantages**	**Disadvantages**
Burp Suite	Powerful manual testing tools, advanced features in Pro.	Most powerful features are paid; Pro version is costly.
Nessus	Excellent for infrastructure and system vulnerability scanning.	Limited web app testing; not specialized for web apps.
OpenVAS	Comprehensive framework for network/system audits.	Not focused on web apps; setup is complex.
OWASP ZAP	Easy to use, great for beginners, active community, automation via scripts, CI/CD support.	May lack advanced scanning depth of commercial tools.

4.2 Advantages of ZAP Tool

For this project, we select **OWASP ZAP**[4] as the tool of choice due to the following advantages:

- **Free and Open Source:** ZAP is fully open source and maintained by OWASP.
- **User-Friendly Interface:** Its GUI and quick-start wizards simplify configuration and usage.
- **Seamless Localhost Support:** ZAP integrates easily with locally hosted applications like Jaltantra.
- **Active and Passive Scanning Modes:** It supports both modes, allowing flexibility during development.
- **Detailed Reports:** Reports are well-structured and severity-ranked, assisting in issue prioritization.
- **OWASP Backing:** With strong community support and frequent updates, ZAP stays current with modern threats.

- **CI Compatibility:** ZAP supports headless mode, enabling integration with CI/CD pipelines.

Given these features, OWASP ZAP is an excellent fit for scanning the Jaltantra application effectively.

4.3 Overview of ZAP Tool

ZAP (Zed Attack Proxy) is a free, open-source security scanner developed by OWASP. It is designed to simulate attack scenarios against websites and discover common vulnerabilities.

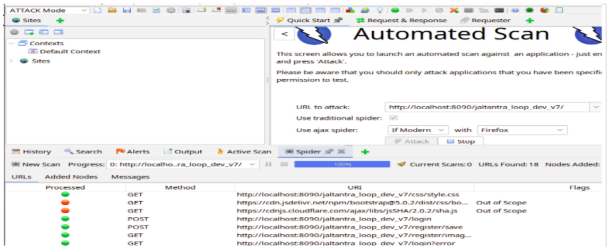

Figure 4.1: Overview of ZAP Tool

ZAP includes:

- Automated vulnerability scanning
- HTTP/HTTPS proxying and interception
- Active and passive scans
- A graphical interface for manual review
- Structured reports with detailed findings

In this project, we scan the local Jaltantra instance at:

`http://localhost:8090/jaltantra_loop_dev_v7/`

We use both traditional and Ajax spiders to crawl the application before initiating an active scan.

4.4 Vulnerabilities Detected

ZAP identifies the following vulnerabilities during our scans:

1. Absence of Anti-CSRF Token

- **URL:** /login
- **Risk:** Medium
- **Description:** Login forms lack CSRF protection, making them vulnerable to CSRF attacks.
- **Solution:**
  - Implement CSRF tokens using Spring Security or OWASP CSRFGuard.
  - Use a nonce in each form and verify it upon submission.
  - Avoid using GET for state-changing operations.

2. Content Security Policy (CSP) Header Not Set

- **URL:** /
- **Risk:** Medium
- **Description:** Missing CSP headers increase risk of XSS and injection attacks.
- **Solution:** Add CSP headers in Spring Security using:

```
http.headers()
    .contentSecurityPolicy("default-src 'self';")
    .and();
```

3. Session ID in URL Rewrite

- **URL:** /login;jsessionid=...
- **Risk:** Medium

- **Description:** Session identifiers in URLs can leak via logs or browser history.
- **Solution:**
  - Store session IDs in cookies.
  - Disable URL-based session tracking in Spring Boot.

4. Cross-Domain JavaScript Source Inclusion

- **URL:** /login
- **Risk:** Low
- **Description:** JavaScript files loaded from third-party CDNs pose injection risks.
- **Solution:**
  - Use trusted CDNs with Subresource Integrity (SRI).
  - Host critical scripts locally when feasible.

Configuration Issues in Code

Current Spring Security configuration disables CSRF protection entirely:

```
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.csrf().disable()
        .authorizeHttpRequests(authorize ->
            authorize.requestMatchers("/", "/images/**", "/css/**",
            "/register/**", "/verify/**").permitAll()
            .anyRequest().authenticated())
        .formLogin(form -> form
            .loginPage("/login")
            .loginProcessingUrl("/login")
            .defaultSuccessUrl("/")
            .permitAll());
}
```

Issue: CSRF is disabled, and no CSP headers are configured, aligning with the risks detected by ZAP.

4.5 SQL Injection (SQLi)

SQL Injection enables attackers to manipulate database queries by injecting malicious SQL into input fields.

What is `sqlmap`?

`sqlmap` is an automated testing tool that detects and exploits SQL injection vulnerabilities across major databases using multiple techniques (boolean-based, time-based, error-based, etc.).

Why Use `sqlmap`?

- Automates payload injection across common vectors.
- Supports database fingerprinting and data extraction.
- Includes tamper scripts to bypass filters and WAFs.
- Open-source and community maintained.

Using `sqlmap`

We use the following command to test the login endpoint:

```
python sqlmap.py \
 -u "http://localhost:8090/jaltantra_loop_dev_v7/login?\
Email=anksuhmitra18@gmail.com&Password=12345678" \
 --level=5 --risk=3 --tamper=space2comment --batch
```

How `sqlmap` Detects SQL Injection

- **True/False Logic:** Modifies parameters to observe behavior for true/false SQL conditions.
- **Error-Based Tests:** Triggers DB errors and checks responses.

- **Time-Based Tests:** Uses delay-based payloads to confirm server-side execution.
- **Fingerprinting:** Identifies DBMS and retrieves structure.

Analysis of the `sqlmap` Output

The scan reports no injectable parameters. This suggests that input sanitization, prepared statements, or WAFs are effectively blocking SQL injection. Further testing with obfuscated payloads and alternate endpoints can provide additional confidence.

Chapter 5

Performance Testing

5.1 Overview

Performance testing validates an application's responsiveness, stability, and scalability under both expected and extreme workloads. For Jaltantra, a water-distribution optimization system, ensuring robust performance is critical to delivering real-time optimization results and maintaining system availability.

The main objectives of performance testing on Jaltantra are:

1. **Ensure Responsiveness:** Measure how quickly the application responds to user requests and optimization jobs under typical load.
2. **Determine Capacity:** Identify the maximum number of concurrent users and optimization tasks the system can handle before performance degrades.
3. **Validate Scalability:** Confirm that Jaltantra scales horizontally (more VUs, more servers) and vertically (more CPU, more memory) without disproportionate drop in performance.
4. **Guarantee Stability:** Verify that the system remains stable over time under sustained load, with no crashes, memory leaks, or thread-pool exhaustion.

When testing Jaltantra, we track:

- **Response Time:** Time taken for each API call or optimization endpoint to return a result.
- **Throughput:** Number of successful requests or optimization runs per second.
- **Error Rate:** Percentage of failed requests or timeouts.
- **CPU & Memory Usage:** Resource consumption on application servers during tests.
- **Garbage Collection Pauses:** Duration and frequency of JVM GC events, to detect memory churn issues.

There are 5 types of performance testing:

Smoke Test: A quick, light load to verify that the system is up and endpoints respond at all.

Load Test: Simulate expected user load (e.g., 50 VUs over 5-10 minutes) to verify response times and throughput.

Stress Test: Push beyond capacity limits (e.g., 400 VUs) to find breaking points and observe failure modes.

Spike Test: Introduce sudden peaks (e.g., jump from 10 to 100 VUs instantly) to evaluate elasticity and auto-scaling.

Soak Test: Run a moderate load (e.g., 50-75 VUs) over an extended period (e.g., 24 hours) to detect memory leaks and degradation.

Comparison of Performance Testing Tools

k6

- **Language Base:** Go + JavaScript (Goja)
- **Scriptability:** Modern JavaScript scripting
- **CI/CD Integration:** Excellent CLI & DevOps support
- **Notable Weakness:** No GUI, limited support for complex browser-based scenarios

Apache JMeter

- **Language Base:** Java
- **Scriptability:** XML-based scripting, optional Groovy support
- **CI/CD Integration:** Legacy integration; needs plugins or wrappers
- **Notable Weakness:** Heavy GUI; not optimized for modern CI workflows

Gatling

- **Language Base:** Scala
- **Scriptability:** Strong Scala-based DSL
- **CI/CD Integration:** Good SBT/Maven integration
- **Notable Weakness:** Requires Scala knowledge; steeper learning curve

Locust

- **Language Base:** Python
- **Scriptability:** Pure Python scripting
- **CI/CD Integration:** Simple CLI and scripting support
- **Notable Weakness:** Not suitable for browser-level testing

Below are Advantages of `k6`:

- **Seamless CI/CD Integration:** Load tests live alongside code, running automatically on each push.
- **Code-First Approach:** Tests written in JavaScript fit naturally into our existing development workflow.
- **Lightweight Execution:** Single binary with minimal dependencies simplifies deployment on build agents.
- **High Concurrency:** Go goroutines enable us to simulate thousands of concurrent optimization requests without needing a large grid of machines.
- **Built-in Metrics & Thresholds:** Easy to set performance budgets and fail builds if response times or error rates exceed limits.
- **Active Community & Extensions:** Regular updates and plugins (e.g., xk6 extensions) help us test WebSockets, gRPC, and other protocols as Jaltantra evolves.

5.2 `k6` – Modern Load Testing Tool

`k6` is an open-source, JavaScript-based load and performance testing tool designed for developers and CI/CD workflows:

- **Standalone Binary:** Runs without extra servers or a UI—just a single executable.
- **Scriptable & Versionable:** Test scripts are plain ES6 JavaScript, stored alongside application code.
- **Developer-Friendly:** Familiar syntax, built-in checks, and easy debugging.
- **Easy CLI & CI Integration:** Integrates with Jenkins, GitHub Actions, GitLab CI, etc.
- **Lightweight & High Performance:** Written in Go, minimal overhead even under thousands of VUs.

Architecture of `k6`

Go & Goja: `k6` itself is implemented in Go. Each Virtual User (VU) is a Go goroutine running your script inside an embedded Goja JavaScript VM[3].

Goroutine Efficiency:

- Tiny, dynamically-resized stacks
- User-space scheduling
- Minimal context for switching
- Efficient blocking

This makes goroutines much cheaper than OS threads, allowing `k6` to spawn thousands of VUs on a single machine.

How `k6` Creates Virtual Users

1. **Script Compilation:** On startup, `k6` parses and compiles your ES6-style script once on the main thread.
2. **Spawning VUs as Goroutines:** For each configured VU, `k6` spins up a separate Go goroutine.
3. **Isolated JS Runtimes:** Inside each goroutine, `k6` creates a fresh Goja JavaScript engine instance.
4. **VU Lifecycle Loop:** Each VU repeatedly invokes your exported function (respecting any `sleep()` calls) in a tight loop.
5. **Networking & Blocking:** HTTP requests (`http.get`, `http.post`, etc.) use Go's `net/http` client under the hood.
6. **Metrics Collection:** Each VU records timing and check metrics locally; a central engine aggregates them in real time for reporting and threshold checks.

5.3 Performance Testing with k6

To ensure the Jaltantra system performs well under different levels of user load, WE use the open-source tool **k6** to perform automated performance testing.

Client-Server Configuration

Client

- **Machine IP:** `10.0.2.15`
- **Role:** Acts as the load generator running the k6 test script.
- **Action:** Sends HTTP requests to the server endpoints to simulate user activity.

Server

- **Machine IP:** `10.129.6.131`
- **Role:** Hosts the Jaltantra application.
- **Port:** `8090`
- **Base URL:** `http://10.129.6.131:8090/jaltantra_loop_dev_v7`

Five types of tests were executed:

Smoke Test

A smoke test is a quick sanity check to confirm that the system and test scripts work correctly under minimal load. It uses a small number of Virtual Users (VUs) for a short duration.

- **Virtual Users:** 1
- **Duration:** 30 seconds
- **Purpose:** Detect any immediate or critical failures

k6 Smoke Test Script:

```
import http from 'k6/http';
import { check, group, sleep } from 'k6';

export let options = {
```

```
  vus: 1,
  duration: '30s',
  thresholds: {
    http_req_duration: ['p(95)<500'],
  },
};

const BASE = 'https://10.129.6.131:8090/jaltantra_loop_dev_v7';

export default function () {
  group('Smoke Test: Public Pages', () => {
    let res;

    res = http.get(`${BASE}/`);
    check(res, {
      'home: status is 200': (r) => r.status === 200,
      'home: body is non-empty': (r) => r.body.length > 100,
    });

    res = http.get(`${BASE}/login`);
    check(res, { 'login page: status is 200': (r) => r.status === 200 });

    res = http.get(`${BASE}/register`);
    check(res, { 'register page: status is 200': (r) => r.status === 200 });

    res = http.get(`${BASE}/loop`);
    check(res, { 'loop: status is 200': (r) => r.status === 200 });

    res = http.get(`${BASE}/branch`);
    check(res, { 'branch: status is 200': (r) => r.status === 200 });
  });

  sleep(1);
}
```

Result: All pages responds with status code 200 and met performance thresholds. Smoke test passes successfully.

Load Test

A load test checks if the application can handle expected production traffic. It increases the number of virtual users and tests the system for a longer duration.

- **Virtual Users:** 50 (peak)
- **Duration:** 5 minutes (1m ramp-up, 3m steady, 1m ramp-down)
- **Purpose:** Ensure system stability and acceptable response time under load

k6 Load Test Script:

```
import http from 'k6/http';
import { check, group, sleep } from 'k6';

export let options =
    { duration: '1m', target: 50 }, // ramp-up to 50 users
    { duration: '3m', target: 50 }, // stay at 50 users
    { duration: '1m', target: 0 },  // ramp-down
  ],
  thresholds: {
    http_req_duration: ['p(95)<800'],  // 95% of requests must finish within 800ms
    checks: ['rate>0.99'],             // less than 1% of checks may fail
  },
};

const BASE = 'https://10.129.6.131:8090/jaltantra_loop_dev_v7';

export default function () {
  group('Public Pages', () => {
    let res = http.get(`${BASE}/`);
    check(res, { 'home is 200': (r) => r.status === 200 });
```

```
  res = http.get(`${BASE}/login`);
  check(res, { 'login page is 200': (r) => r.status === 200 });

  res = http.get(`${BASE}/register`);
  check(res, { 'register page is 200': (r) => r.status === 200 });
});

group('Login', () => {
  const payload = {
    username: __ENV.USERNAME,
    password: __ENV.PASSWORD,
  };
  const params = {
    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    redirects: 0,
  };
  let loginRes = http.post(`${BASE}/login`, payload, params);
  check(loginRes, {
    'login status is 302': (r) => r.status === 302,
    'has session cookie': (r) =>
      !!(r.cookies['JSESSIONID'] || r.cookies['SESSION']),
  });
});

group('Authenticated Pages', () => {
  let res = http.get(`${BASE}/loop`);
  check(res, { 'loop is 200': (r) => r.status === 200 });

  res = http.get(`${BASE}/branch`);
  check(res, { 'branch is 200': (r) => r.status === 200 });

  // Additional endpoints can be tested here
});
```

```
  sleep(Math.random() * 3 + 1);
}
```

Result: The load test is successfully completed without any major failures. Key observations:

- 95% of HTTP requests completes within 800 milliseconds.
- Login operations returns status 302 as expected (redirect after success).
- Session cookies like `JSESSIONID` were properly set for authenticated users.
- No endpoint returned error status codes (e.g., 4xx or 5xx).
- The application remains stable throughout the 5-minute test.

Stress Testing

Stress testing is used to evaluate how the system behaves when pushed beyond normal or peak load conditions. It helps determine the breaking point of the system by gradually increasing virtual users (VUs) until performance degrades or service level agreements (SLAs) are violated.

Stress tests are typically run:

- After regular load tests are passed.
- Before go-live events such as deadlines or high-traffic periods.

Stress Test Plan

- **Ramp-up:** Gradually increase VUs to extreme levels.
- **Hold:** Maintain high traffic to observe system stability.
- **Ramp-down (optional):** Bring users down slowly.

k6 Stress Test Script

```
import http from 'k6/http';
import { check, group, sleep } from 'k6';

export let options = {
  stages: [
    { duration: '2m', target: 50 },
    { duration: '3m', target: 100 },
    { duration: '2m', target: 200 },
    { duration: '3m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<1200'],
    checks: ['rate>0.98'],
  },
};

const BASE = 'https://10.129.6.131:8090/jaltantra_loop_dev_v7';

export default function () {
  group('Public Endpoints', () => {
    let res = http.get(`${BASE}/`);
    check(res, { 'home status 200': (r) => r.status === 200 });
    res = http.get(`${BASE}/login`);
    check(res, { 'login page 200': (r) => r.status === 200 });
  });

  group('Login', () => {
    const payload = {
      username: __ENV.USERNAME,
      password: __ENV.PASSWORD,
    };
    const params = {
```

```
      headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
      redirects: 0,
    };
    let loginRes = http.post(`${BASE}/login`, payload, params);
    check(loginRes, {
      'login status 302': (r) => r.status === 302,
      'session cookie set': (r) =>
        !!(r.cookies['JSESSIONID'] || r.cookies['SESSION']),
    });
  });

  group('Authenticated Endpoints', () => {
    let res = http.get(`${BASE}/loop`);
    check(res, { 'loop 200': (r) => r.status === 200 });

    res = http.get(`${BASE}/branch`);
    check(res, { 'branch 200': (r) => r.status === 200 });
  });

  sleep(Math.random() * 2 + 1);
}
```

Stress Test Results

The following table shows how system latency and throughput changes as the number of virtual users increases. The SLA target is set at 95% of requests completing within 1 second.

Observations

- The application performs well up to 430 virtual users, with low latency and high request throughput.
- At 450 users, 450 users but passes on retry—likely due to cold start or network hiccups.

Table 5.1: Max Users vs. Success Rate during Stress Testing

Max Users	**Success Rate (%)**
100	100.00
400	100.00
425	100.00
430	99.99
450	99.98
500	99.97
1000	99.59
1500	99.52
1900	98.63
1935	98.07
1940	97.97
1950	97.44
2000	97.38

- Beyond 1940 users, latency increases significantly and SLA targets are violated.

The stress test helps determine the upper limits of the Jaltantra system under extreme conditions. The system performs reliably up to 1000–2000 users, beyond which performance degrades. This insight can guide future efforts to optimize backend performance, database efficiency, and server capacity. These tests also help define safe concurrency limits for production deployment.

Spike Testing

Spike testing validates how the system responds to sudden traffic surges. This simulates real-world scenarios like flash sales, admission portals, or ticketing platforms.

Spike Test Characteristics

- **Instant Ramp-Up:** Load spikes quickly from normal to peak traffic.
- **Short Hold:** System handles the spike for a brief period.
- **Quick Recovery:** Load drops rapidly to baseline to observe recovery.

k6 Spike Test Script

```
export let options = {
  stages: [
```

```
    { duration: '1m', target: 10 },     // baseline
    { duration: '30s', target: 300 },   // spike up
    { duration: '1m', target: 300 },    // hold spike
    { duration: '30s', target: 10 },    // spike down
    { duration: '1m', target: 10 },     // recovery
  ],
  thresholds: {
    http_req_duration: ['p(95)<1000'],
    checks: ['rate>0.99'],
  },
};
```

Spike Test Results

The system handles the spike without any errors or memory issues. Even under sudden surge from 10 to 300 VUs, latency remains within acceptable range (p95 = 20.7 ms). The system recovers gracefully after the spike phase. This shows Jaltantra's resilience under unexpected traffic bursts.

Soak Testing

Soak testing verifies system stability over long durations. It helps detect slow memory leaks, resource exhaustion, or performance degradation that short tests may not reveal.

Soak Test Characteristics

- Long-duration test (e.g., 1+ hours).
- Constant traffic at moderate levels.
- Observes trends in latency, memory, and error rates over time.

k6 Soak Test Configuration

This test was run with 50 Virtual Users for 1 hour, sending data to InfluxDB + Grafana for monitoring.

```
stages: [
  { duration: '5m', target: 50 },
  { duration: '1h', target: 50 },
  { duration: '5m', target: 0 },
],
thresholds: {
  http_req_duration: ['p(99)<800'],
  checks: ['rate>0.995'],
  vus: ['value>=50'],
},
```

Soak Test Results-1 hour @50 user

Table 5.2: Soak Test Results-1 hour @50 user

Metric	**Value**
Virtual Users (Configured)	50
Actual Max VUs	50
Requests per Second	8.23
Failure Rate (%)	0.01
Check Success Rate (%)	100
Errors Observed	Login timeout, connection reset by peer

Soak Test Results-2 hour @75 user

Table 5.3: Soak Test Results-2 hour @75 user

Metric	**Value**
Virtual Users (Configured)	75
Virtual Users (Achieved)	75
Request Rate (avg)	41.70 req/s
Check Success Rate (%)	99.75%
Failure Rate (%)	0.24%
Errors Observed	Multiple endpoint response mismatches

Observation

The system performs reliably over the 1-hour soak test. Memory usage remains flat, and no signs of heap leaks or GC pauses are detected. Overall metrics remains within defined thresholds.

5.4 Real-time Monitoring with Prometheus and Grafana

To monitor system performance metrics in real time during load and soak tests, we integrate **Prometheus** and **Grafana** into the Jaltantra backend. This setup enables live tracking of CPU usage, memory, garbage collection (GC), HTTP latency, thread pools, and other system-level metrics during testing.

Why Use Prometheus and Grafana?

- **Prometheus** is an open-source time-series database and metrics scraper. It pulls numerical metrics over HTTP at regular intervals from specified targets.
- **Grafana** is an open-source visualization and alerting platform that reads data from Prometheus and other sources. It supports rich dashboards, live panels, annotations, and alerts.

The combined use of these tools provides:

- Real-time visibility during performance testing.
- Detection of JVM heap or GC bottlenecks.
- Historical trend tracking for capacity planning.
- Customizable dashboards to share results with stakeholders.

Setup: Spring Boot + Prometheus Integration

To expose metrics from the Jaltantra backend, we add the following Maven dependencies in `pom.xml`:

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
 <groupId>io.micrometer</groupId>
```

```
 <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

In `application.properties`, we include:

```
server.servlet.context-path=/jaltantra_loop_dev_v7
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.server.base-path=/actuator
```

We also update Spring Security to allow access to Prometheus metrics by modifying the `SecurityFilterChain`:

```
http.csrf().disable()
 .authorizeHttpRequests(authorize -> authorize
 .requestMatchers("/", "/images/**", "/css/**",
"/register/**", "/verify/**",
"/actuator/prometheus").permitAll()
 .anyRequest().authenticated());
```

Prometheus Configuration

In the Prometheus configuration file (`prometheus.yml`), we define the following scrape job:

```
global:
 scrape_interval: 15s

scrape_configs:
 - job_name: 'jaltantra'
 metrics_path: '/jaltantra_loop_dev_v7/actuator/prometheus'
 static_configs:
 - targets: ['localhost:8090']
```

Grafana Setup

In the Grafana UI:

1. We add Prometheus as a data source: `http://localhost:9090`.
2. We import dashboard ID **4701** (Spring Boot Actuator Metrics).
3. We map the data source to the Prometheus instance.

Monitoring Workflow

The real-time monitoring workflow proceeds as follows:

- We start Prometheus and Grafana servers.
- We run the performance test using: `k6 run perf.js`.
- We observe live metrics in Grafana dashboards.
- We monitor JVM memory usage, GC activity, request latencies, and thread saturation.

This setup provides deep insights into system behavior under varying test loads, helping us detect hidden issues such as memory leaks, slow heap growth, or inefficient thread utilization.

Chapter 6

End to End Testing

6.1 End-to-End (E2E) Testing with Selenium

End-to-End (E2E) testing is a type of testing that simulates a real user's journey through the application. It validates that all layers of the system—frontend, backend, database, file handling, and authentication—work together correctly. E2E testing helps detect issues that unit or integration tests may miss by verifying full business workflows from start to finish.

Table 6.1: Comparison of E2E Testing Tools

Tool	**Language Support**	**Framework Compatibility**	**Browser Coverage**
Cypress	JavaScript, TypeScript	Requires Node.js environment	Chromium-based browsers, Firefox (No Internet Explorer)
Playwright	JavaScript, TypeScript, Python, .NET	Requires Node.js environment	Chrome, Firefox, WebKit (covers Safari-like rendering)
TestCafe	JavaScript, TypeScript	Requires Node.js environment	Chrome, Firefox, Edge
Selenium	Java, Python, C#, Ruby, etc.	Works well with Spring Boot + Maven/Gradle	Chrome, Firefox, Safari, Internet Explorer, Edge, Mobile browsers

For this project, Selenium was selected for the following reasons:

- **Maturity and Community Support:** Over 15 years of development and widespread adoption.

- **Cross-Browser Testing:** Supports Chrome, Firefox, Safari, Edge, etc.
- **Language Flexibility:** Allows writing tests in Java (used here), Python, JavaScript, and more.
- **Integration Support:** Easily integrates with TestNG, Maven, and reporting tools.

6.2 Project Configuration

The following dependencies were added to `pom.xml`:

```
<!-- Selenium WebDriver -->
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.21.0</version>
</dependency>

<!-- TestNG -->
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.9.0</version>
  <scope>test</scope>
</dependency>

<!-- Surefire for running TestNG -->
<dependency>
  <groupId>org.apache.maven.surefire</groupId>
  <artifactId>surefire-testng</artifactId>
  <version>3.1.2</version>
</dependency>
```

A `testng.xml` suite was configured as:

```
<suite name="Jaltantra E2E Suite">
```

```
  <test name="EndtoEndTest">
    <classes>
      <class name="com.hkshenoy.jaltantraloopsb.JaltantraEndToEndTest"/>
    </classes>
  </test>
</suite>
```

6.3 Overview of E2E Test Case

The test case simulates a full user workflow on the Jaltantra system:

1. **User Registration:** The test opens the registration form and fills user data dynamically using unique email addresses.
2. **Login:** It logs in with the newly registered credentials and verifies successful authentication.
3. **Application Navigation:** Navigates to the "Branch Optimizer" tool through the sidebar.
4. **Excel Upload:** Uploads an Excel file (`*.xls`) containing pipe and node data via file input.
5. **Optimization Trigger:** Executes the optimization process via sidebar navigation and confirms redirection to results.
6. **Download and Compare Output:** Downloads the optimized output file and compares it with a pre-stored expected output.
7. **Cleanup:** Ensures browser teardown and removes stale downloaded files after test completion.

The test leverages JavaScript execution for interacting with dynamic sidebar elements and stubbing popup confirmations. It uses custom download handling, file comparison, and wait strategies to ensure stability and reproducibility.

Below are benefits of test automation:

- Fully simulates actual user interactions.
- Verifies correctness of both UI and backend workflows.
- Detects integration issues between frontend, file handlers, and optimization engine.
- Provides regression safety for future UI or backend changes.

6.4 Conclusion

Selenium-based end-to-end (E2E) testing provides a comprehensive way to validate Jaltantra's complete user journey. The test automation covers registration, login, data submission, optimization, and result verification—ensuring that the platform behaves as expected under real usage scenarios. This test is now part of the regression suite and can be expanded to support additional user roles and workflows in future iterations.

Chapter 7

Continuous Integration using GitHub Actions

7.1 Introduction to Continuous Integration (CI)

Continuous Integration (CI) is a development practice in which code changes are automatically built, tested, and validated every time they are pushed to the source code repository. It ensures that integration problems are detected early and that broken code is not merged into the main branch.

A CI system automates:

- Code checkout
- Build and dependency management
- Test execution (unit, integration, E2E)
- Artifact packaging
- Notifications of build results

There are several popular CI tools used by developers and DevOps teams:

- **Jenkins:** A widely used open-source CI tool with a rich plugin ecosystem. Requires manual setup and server maintenance.

- **Travis CI:** A cloud-based CI service integrated with GitHub. Offers simplicity but limited free usage for private repos.
- **CircleCI:** Known for fast parallel builds and Docker-native pipelines. Well-suited for microservices.
- **GitLab CI/CD:** Built into GitLab, with advanced pipeline management and self-hosted support.
- **GitHub Actions:** A relatively newer CI/CD service tightly integrated into GitHub, allowing native workflows through '.yml' files.

Advantages of GitHub Actions

The Jaltantra project is hosted on GitHub, making GitHub Actions the most logical and efficient choice. The key reasons include:

- **Native GitHub Integration:** Works directly with GitHub repositories and pull requests.
- **Configuration as Code:** All CI logic is written as YAML files inside the repository.
- **Free Tier for Public Repos:** Ideal for open-source academic projects.
- **Matrix Builds:** Allows testing across multiple environments (JDK versions, OS, etc.).
- **No Additional Setup:** Fully managed runners with Linux, Windows, and macOS support.

7.2 GitHub Actions Configuration for Jaltantra

To set up CI, I created a workflow file under: `.github/workflows/java-ci.yml`

Below is the workflow configuration used:

```
name: Java CI

on:
```

```
  push:
    branches:
      - main
      - master
      - ankush2
  pull_request:
    branches:
      - main
      - master
      - ankush2

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    # 1. Check out the code
    - name: Checkout repository
      uses: actions/checkout@v3

    # 2. Set up Java 17 using Temurin distribution
    - name: Set up JDK 17
      uses: actions/setup-java@v3
      with:
        distribution: temurin
        java-version: 17

    # 3. Build the project and run tests
    - name: Build & Test
      run: ./mvnw clean package
```

7.3 CI Workflow Explanation

The configured workflow does the following:

- Triggers on pushes or pull requests to the `main`, `master`, or `ankush2` branches.
- Uses an Ubuntu runner to execute jobs.
- Checks out the latest version of the code.
- Sets up JDK 17 using Temurin.
- Runs the full Maven lifecycle (‘clean‘ and ‘package‘) to build the code and execute all unit and integration tests.

Benefits for the Jaltantra Project

- Every code push is validated automatically with build and test feedback.
- Prevents broken code from being merged into main branches.
- Reduces manual testing effort and ensures test coverage is always maintained.
- Provides traceability with logs for each build and test run.
- Enables consistent and repeatable testing across environments.

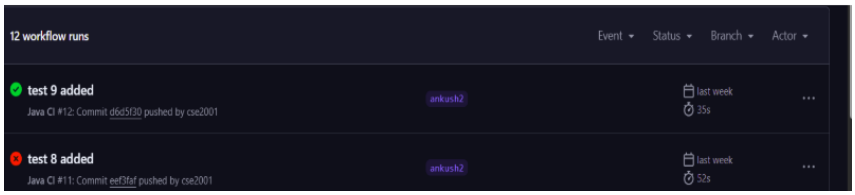

Figure 7.1: Results when all test cases passes and when wrong test occurs

7.4 Conclusion

Integrating GitHub Actions into the Jaltantra repository introduced an efficient and maintainable CI pipeline. It enforces discipline in the development workflow by ensuring that all code changes are verified through automated builds and tests. The CI pipeline is fast, cost-effective, and scales with the growing complexity of the project. This setup forms a foundation for future enhancements like deployment, code quality analysis, and performance regression tracking.

Chapter 8

Jaltantra Handbook and Documentation

8.1 Introduction

As Jaltantra matured into a complex and multi-module system, the need for structured documentation became critical. To bridge the knowledge gap and ensure maintainability, a comprehensive technical guide—referred to as the **Jaltantra Handbook**—was developed. It serves as the authoritative reference for developers, testers, maintainers, and deployers interacting with the platform.

8.2 Why Documentation Was Necessary

Before this effort, project knowledge was scattered across various informal channels such as legacy code comments, shared scripts, or verbal communication. This lack of formal documentation posed several issues:

- Long onboarding time for new contributors
- Risk of misconfiguration during deployment
- High dependence on individual developers
- Limited visibility into system internals and API usage

The introduction of the Jaltantra Handbook was a deliberate step to formalize and consolidate all this knowledge into an accessible and versioned resource.

8.3 Contribution to the Handbook

As part of our contribution to the development and documentation of the Jaltantra project, we author two core chapters in the official handbook: **Branch Optimizer** and **Work Done So Far in Jaltantra**.

Chapter: Branch Optimizer

In this chapter, we present a comprehensive overview of the architecture, components, and workflow of the branch optimization module within Jaltantra. The chapter covers:

- A detailed breakdown of the project structure including controllers, helper utilities, optimization algorithms, and data structures.
- Explanation of each Java class involved in the pipeline—from input file handling to result processing.
- Focus on critical files such as `BranchController.java`, `BranchOptimizer.java`, `Problem.java`, and `Result.java` that drive the optimization logic for branched water distribution networks.
- A visual workflow diagram to aid understanding of the optimization process.
- External references to test files and resources for validation and experimentation.

This chapter enables developers and users to clearly understand the internals of branch optimization, and serves as a guide to extend or debug the system effectively.

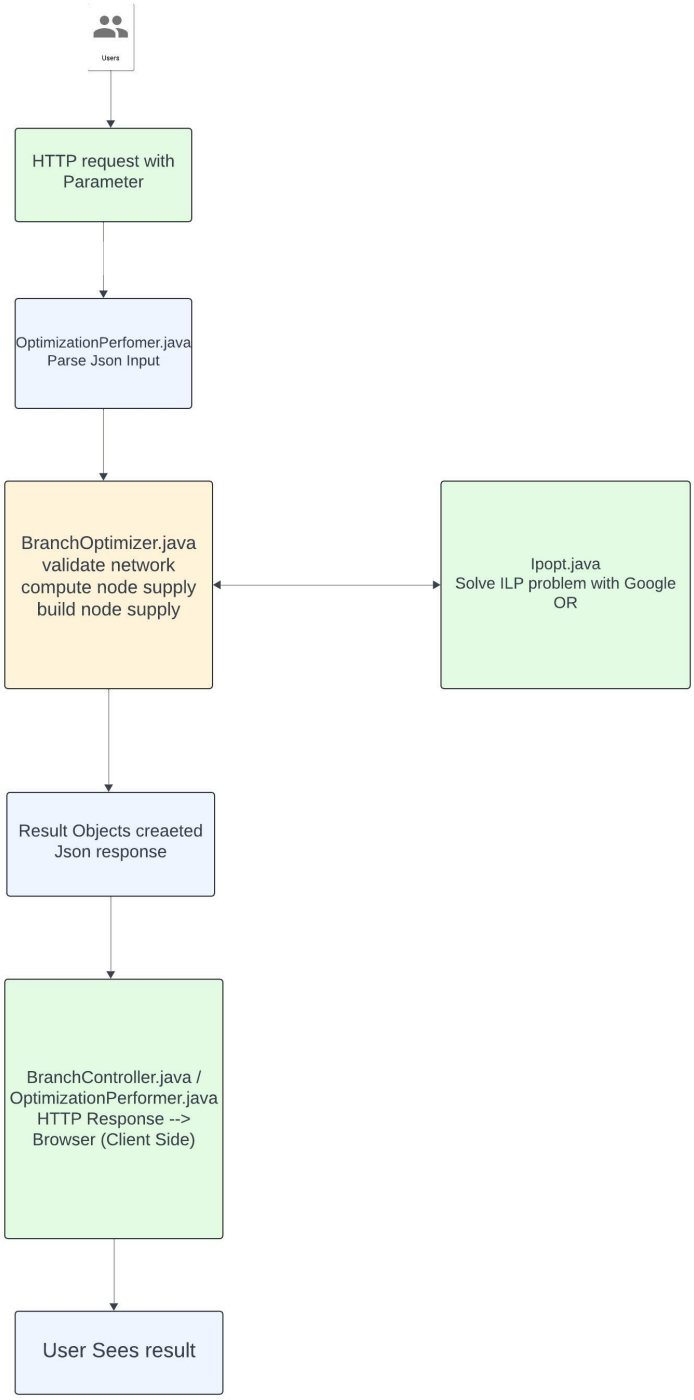

Figure 8.1: Work flow of Branch Optimizer

Chapter: Work Done So Far in Jaltantra

We also compile and document the efforts of all major contributors to the Jaltantra platform over multiple development phases. This includes:

- Technical summaries of foundational work done by Dr. Nikhil Hooda, including ILP formulations and ESR modeling.
- Backend and solver integration efforts by Fenil Mehta to scale loop optimization using Octeract and Baron.
- System infrastructure development and REST API integration by Siddhant Singh for loop-based optimization.
- Loop optimization modeling and UI contributions by Shantanu Mapari for urban networks.
- Spring Boot migration, deployment automation, and security enhancements led by Harikrishna Shenoy.

By curating this information into a single chapter, we ensure long-term visibility of individual efforts and provide a historical context for ongoing development in Jaltantra.

Together, these chapters contribute significantly to the maintainability, onboarding, and scalability of the Jaltantra platform, ensuring that institutional knowledge is preserved and accessible to all future collaborators.

8.4 Impact on the Project

The availability of formal documentation has already made a significant impact:

- **Reduced Onboarding Time:** New contributors can now understand and start working on the codebase much faster.
- **Reliable Deployment:** Detailed setup and deployment guides eliminate guesswork.
- **Debug Efficiency:** Common failure cases are documented, reducing time-to-resolution.

- **Improved Collaboration:** Everyone works with the same assumptions and knowledge.

8.5 Conclusion

The Jaltantra Handbook transforms previously scattered tribal knowledge into an organized, accessible, and evolving reference. It brings clarity, stability, and scalability to both the development and operational sides of the project. By prioritizing documentation as a core deliverable, the project is now better positioned for sustainable growth and collaborative contribution.

Chapter 9

Summary and Conclusions

9.1 Contribution

This project makes several key contributions to the Jaltantra platform:

- Develops the first formal **performance testing framework** using `k6`, covering Smoke, Load, Stress, Spike, and Soak tests to assess system responsiveness and stability.
- Conducts **end-to-end security testing** with OWASP ZAP, identifying vulnerabilities in authentication, session management, and API protection.
- Integrates **Continuous Integration (CI)** using GitHub Actions, enabling automated build and test workflows directly from the repository.
- Enables **real-time system monitoring** with Prometheus and Grafana, visualizing JVM metrics, latency, heap usage, and request throughput.
- Creates a robust **end-to-end Selenium-based test suite** simulating real user workflows from registration to output verification.
- Authors two core chapters in the **Jaltantra Handbook**: *Branch Optimizer* and *Work Done So Far in Jaltantra*, documenting the system architecture, optimization flow, project history, and individual contributions.
- Adds **test files** in google drive.

9.2 Future Work

The following directions are proposed for continued improvement and scalability of the Jaltantra platform:

- **Extend CI to Continuous Deployment (CD)** using Docker or Kubernetes for automated environment provisioning and updates.
- **Parallize test cases** to reduce time using threads.
- **Integrate gurobi solvers** to loop optimizer. .
- **Create test cases environment** for loop.
- **Fix anti-csrf token and csp header** so that Application is not vulnerable to CSRF and XSS attack.

Bibliography

[1] N. Hooda and O. Damani, "JalTantra: A System for the Design and Optimization of Rural Piped Water Networks," *INFORMS Journal*, 2019, 49(6):447–458, 2019.

[2] R. Johnson and J. Hoeller, "Spring Framework Documentation," Jan. 2024.

[3] k6 – Modern Load Testing Tool. *Grafana Labs*. Available at: `https://k6.io/`.

[4] OWASP ZAP – Zed Attack Proxy. *OWASP Foundation*. Available at: `https://www.zaproxy.org/`.

[5] GitHub Actions – Automate your workflow. *GitHub*. Available at: `https://github.com/features/actions`.

[6] Selenium Project. *Selenium: Browser Automation*. Available at: `https://www.selenium.dev/`.

[7] JetBrains. *IntelliJ Support Request 5446484*. Available at: `https://intellij-support.jetbrains.com/hc/en-us/requests/5446484`.

[8] Oracle Corporation. *MySQL Documentation*. Available at: `https://dev.mysql.com/doc/`.

Acknowledgment

I would like to express my deepest gratitude to my project guide, **Prof. Om Damani**, for his invaluable guidance, constant support, and insightful feedback throughout the course of this work. His mentorship has been instrumental in shaping the direction and outcome of this project.

I am also thankful to my classmates **Sayantan Biswas** and **Utsav Manani** for their collaboration, discussions, and the shared journey that enriched my learning experience.

Special thanks to my seniors **Shantanu Mapari** and **Harikrishna Shenoy** for their technical support, encouragement, and for laying down a solid foundation that helped me navigate the complexities of the system.

Their contributions and support have been pivotal to the successful completion of this thesis.